Unidad 1: Practica Semana2



Materia:

Base de Datos III (BDD3T4)

Estudiantes:

Julio Cesar Cruz Kübber

Diego Emmanuel Gorostiaga Gonzales

Dario Haziel Tambo Parra

Josue Carlos Soliz Mamani

Carrera:

Ingeniería en Sistemas Informáticos

Docente:

Romel Soliz Vargas

Fecha:

31 de agosto de 2026

Tablas de Objetos, Referencias y Colecciones

Proyecto Integrador — Avance 1, Semana 2 — Caso EduConecta — Grupo 3

1. Alcance

La Semana 1 dejó definidos y verificados con `DESCRIBE` los tipos `TIPO_DIRECCION` y `TIPO_ESTUDIANTE`. Este documento acompaña al script `educonecta_semana2.sql` y cubre las actividades 7 a 11 de la Sección 6 de la guía: la tabla de objetos principal, la colección del catálogo con su justificación escrita, la tabla transaccional conectada por `REF`, los datos de prueba y las cuatro operaciones CRUD.

2. Tabla de objetos principal

La entidad principal se materializa con `CREATE TABLE ... OF`, de modo que cada fila es en sí misma un objeto y no una fila con una columna de tipo objeto. La razón es funcional antes que estética: `REF()` opera únicamente sobre filas de tablas de objetos, pues sólo éstas poseen un identificador de objeto (OID) referenciable. Como el modelo exige que la tabla transaccional apunte al estudiante con `REF`, cualquier otro patrón de almacenamiento sería incapaz de satisfacer el requerimiento.

```
CREATE TABLE TABLA_ESTUDIANTES OF TIPO_ESTUDIANTE (
    cod_estudiante PRIMARY KEY
);
```

La clave primaria se declara sobre `cod_estudiante`, el código institucional. Al ser un identificador que ya existe en el dominio del negocio se evita introducir una clave técnica artificial, y la restricción se declara directamente sobre el atributo del tipo, que en una tabla de objetos se comporta como una columna.

3. La colección del catálogo: justificación de la elección

El atributo variable del caso son los módulos de un curso, y se modelan como **Nested Table** de objetos `TIPO_MODULO`, no como VARRAY. El criterio aplicado es el visto en clase: si el límite superior de la colección está fijado de antemano por una regla de negocio, corresponde un VARRAY; si depende del crecimiento de los datos, corresponde una Nested Table.

El número de módulos de un curso pertenece al segundo caso. El propio enunciado describe un catálogo «de estructura variable» que crece a medida que se publica contenido, y un mismo curso puede reestructurarse de cuatro a doce módulos entre versiones del plan de estudios. Al no existir cota superior conocida, un VARRAY obligaría a fijar un número arbitrario y a recrear el tipo cada vez que ese número se excediera, invalidando todo lo que dependa de él.

La decisión tiene además dos consecuencias operativas favorables. La Nested Table se almacena fuera de línea, en su propio segmento declarado con `STORE AS`, de modo que el tamaño de la colección no penaliza la lectura de la fila del curso; y admite DML directo sobre elementos individuales a través de `TABLE(...)`, lo que permite corregir un solo módulo sin reescribir la colección completa. Un VARRAY no ofrece ninguna de las dos: se almacena en línea con la fila y sólo puede reemplazarse entero.

El razonamiento inverso también es válido y conviene dejarlo explícito: si el atributo variable hubiera sido la lista de modalidades habilitadas por la institución —un conjunto cerrado de tres valores en el que además el orden expresa preferencia—, el VARRAY habría sido la elección correcta. En el modelo entregado la modalidad es un atributo escalar, precisamente porque no es variable.

3.1. Los prerrequisitos como referencias entre objetos

Los prerrequisitos no se guardan como copias del código del curso, sino como una colección de `REF TIPO_CURSO`: son relaciones entre objetos del propio catálogo, de modo que se navegan con `DEREF` en lugar de reunirse por valor de clave. Su cardinalidad tampoco está acotada, por lo que se emplea de nuevo una Nested Table. Como `TIPO_PRERREQUISITOS_TAB` referencia a `TIPO_CURSO` y `TIPO_CURSO` contiene a `TIPO_PRERREQUISITOS_TAB`, existe una dependencia circular que Oracle resuelve declarando primero el tipo de forma incompleta.

```
CREATE TYPE TIPO_CURSO;
/
CREATE OR REPLACE TYPE TIPO_PRERREQUISITOS_TAB AS TABLE OF REF TIPO_CURSO;
/
CREATE OR REPLACE TYPE TIPO_CURSO AS OBJECT ( ... );
/
```

```
CREATE TABLE TABLA_CURSOS OF TIPO_CURSO ( cod_curso PRIMARY KEY )
NESTED TABLE modulos STORE AS cursos_modulos_tab
NESTED TABLE prerrequisitos STORE AS cursos_prerrequisitos_tab;
```

TIPO_CURSO incluye además el atributo cupo_maximo (BDD-101: 3, PRG-102: 2, BDD-301: 3), la cantidad de matrículas en curso que admite cada asignatura. El Avance 2 lo usa para impedir que un curso se llene por encima de su cupo.

4. Tabla transaccional conectada por REF

La matrícula es la transacción central del caso y vincula dos entidades distintas mediante dos referencias separadas: `ref_estudiante` de tipo `REF TIPO_ESTUDIANTE` y `ref_curso` de tipo `REF TIPO_CURSO`. Incorpora además el periodo, la beca, el avance y la nota final, que son los datos propios de la transacción y no de ninguna de las dos entidades referenciadas.

```
CREATE TABLE TABLA_MATRICULAS OF TIPO_MATRICULA (
    cod_matricula PRIMARY KEY
);
```

`REF` crea cada referencia al insertar, tomando el objeto de su propia tabla (REF(e) y REF(c) en el INSERT … SELECT), y DEREF la navega en las consultas; así una matrícula solo puede apuntar a TABLA_ESTUDIANTES y a TABLA_CURSOS. Una referencia no sustituye, en cambio, a una clave foránea: si se elimina la fila apuntada, la referencia sobrevive apuntando a un OID inexistente. Ese riesgo se comprueba en el script con `IS DANGLING`.

5. Datos de prueba

Se insertaron tres filas en cada tabla, con la colección de módulos poblada en los tres cursos y con más de un elemento en todos ellos. Los objetos se insertan siempre invocando el constructor del tipo, nunca columna por columna, para que las validaciones definidas en la Semana 1 se ejecuten. El país del segundo estudiante se escribe con tilde en el `INSERT` («Perú») y se almacena normalizado como PERU: es el efecto de la validación del constructor de `TIPO_DIRECCION`.

cod_estudiante	Estudiante	Ciudad	País	Nivel	Sede
EST-2026-001	Rojas Villarroel, Camila Andrea	Cochabamba	BOLIVIA	PREGRADO	Sede Bolivia
EST-2026-002	Quispe Ramos, Mateo Alonso	Lima	PERU	POSGRADO	Sede Peru
EST-2026-003	Ferreyra Duarte, Valentina	Rosario	ARGENTINA	PREGRADO	Sede Argentina

Tabla 1. TABLA_ESTUDIANTES.

cod_curso	Nombre	Créd.	Modalidad	Módulos de la colección (orden – horas)	Prerreq.
BDD-101	Fundamentos de Bases de Datos	4	VIRTUAL	1 Modelo E-R (20) · 2 Algebra relacional y SQL (28) · 3 Normalizacion hasta 3FN (16)	—
PRG-102	Programacion Orientada a Objetos	5	ASINCRONICO	1 Clases, objetos y encapsulamiento (24) · 2 Herencia y polimorfismo (30)	—
BDD-301	Bases de Datos Objeto-Relacionales	6	VIRTUAL	1 Tipos de objeto y constructores (18) · 2 Tablas de objetos y REF (22) · 3 Colecciones VARRAY y Nested Table (26) · 4 Consultas objeto-relacionales (14)	BDD-101, PRG-102

Tabla 2. TABLA_CURSOS, con la colección anidada poblada y los prerrequisitos resueltos.

cod_matricula	Estudiante	Curso	Periodo	Beca	Avance	Nota	Estado
MAT-2026-	EST-2026-	BDD-	2026-2	EXCELENCIA 40%	80	—	EN_CURSO

0001	001	301					
MAT-2026-0002	EST-2026-002	PRG-102	2026-2	SIN_BECA 0%	30	—	EN_CURSO
MAT-2026-0003	EST-2026-003	BDD-101	2026-1	SOCIOECONOMICA 25%	100	78	APROBADO

Tabla 3. TABLA_MATRICULAS, con sus dos referencias resueltas mediante DEREF.

6. Operaciones CRUD

Operación	Alcance ejercitado sobre el esquema propio
CREATE	Alta de un estudiante mediante el constructor del tipo; alta de un módulo dentro de una colección ya existente con `INSERT INTO TABLE(...)`; alta de una matrícula que toma sus dos referencias con `REF()`.
READ	Acceso al objeto anidado con notación de punto y a los métodos propios del tipo; navegación por referencia con `DEREF`; aplanado de la colección con `TABLE()`, en listado y en agregación.
UPDATE	Modificación de un atributo simple; de un atributo dentro del objeto anidado; y de un elemento individual de la Nested Table sin reescribir la colección.
DELETE	Baja de un elemento de la colección; baja de una fila completa; comprobación del riesgo de integridad con `IS DANGLING` entre `SAVEPOINT` y `ROLLBACK TO`.

7. Consultas de verificación con DEREF y TABLE()

Navegación desde la tabla transaccional hacia sus dos entidades referenciadas, sin ningún `JOIN`, y aplanado de la colección anidada para calcular la carga horaria real de cada curso:

```
SELECT m.cod_matricula, DEREF(m.ref_estudiante).apellidos AS apellidos,
       DEREF(m.ref_curso).nombre AS curso, DEREF(m.ref_curso).creditos AS creditos
FROM TABLA_MATRICULAS m ORDER BY m.cod_matricula;

SELECT c.cod_curso, COUNT(*) AS modulos, SUM(m.horas_academicas) AS horas_totales
FROM TABLA_CURSOS c, TABLE(c.modulos) m GROUP BY c.cod_curso ORDER BY c.cod_curso;
```

Resolución de la colección de referencias, donde `TABLE()` y `DEREF` actúan juntos. `COLUMN_VALUE` es la pseudo-columna con la que Oracle expone los elementos de una Nested Table de tipo escalar, como es el caso de una colección de referencias:

```
SELECT c.cod_curso, DEREF(p.COLUMN_VALUE).cod_curso AS cod_prerrequisito,
       DEREF(p.COLUMN_VALUE).nombre AS prerrequisito
FROM TABLA_CURSOS c, TABLE(c.prerrequisitos) p ORDER BY c.cod_curso, cod_prerrequisito;
```

Comprobación del riesgo de integridad de las referencias. Oracle no propaga la eliminación a través de un `REF`: la matrícula sobrevive apuntando a un OID inexistente y `DEREF` devuelve NULL sin lanzar error, de modo que el problema pasaría inadvertido hasta la emisión de un reporte. La prueba se revierte para no alterar los datos.

```
SAVEPOINT antes_de_prueba_dangling;
DELETE FROM TABLA_ESTUDIANTES WHERE cod_estudiante = 'EST-2026-003';
SELECT m.cod_matricula, m.periodo, m.estado FROM TABLA_MATRICULAS m
WHERE m.ref_estudiante IS DANGLING;
ROLLBACK TO SAVEPOINT antes_de_prueba_dangling;
```